



# Hooks, guardrails, евалуације и завршни QA

Извршиве политике, тестови и евалуациони сценарији за контролисан AI радни ток.

## Извршива правила за обавезне политике

Обавезна правила спроводе се кодом и тестовима кад год је то могуће.

**Зато solution садржи `EquipmentReservation.Guardrails` и `NUnit` тестове његових политика.**

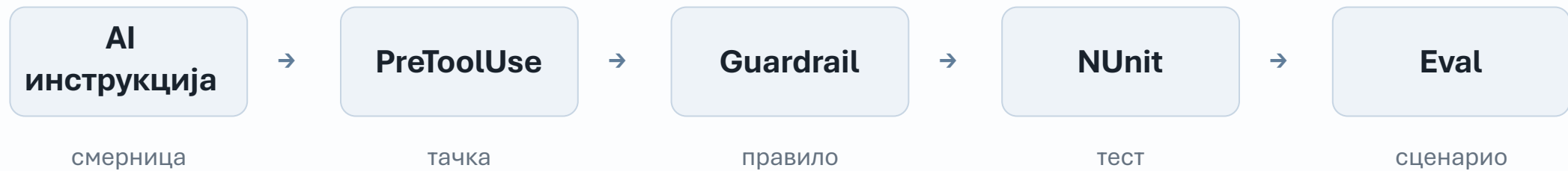
---

### НАПОМЕНА

**Guardrail је извршива политика за контролу агентског тока.**

## Хеуристичка и детерминистичка заштита

Заштита се примењује пре позива алата, током извршења и пре прихватања резултата.



### НАПОМЕНА

Обавезна правила спроводе се детерминистичком провером.

## Интерфејс за guardrail политику

ОСР и DIP омогућавају проширење политика без измене централног тока.

**GuardrailEvaluator** зависи од колекције апстракција, па се нова политика додаје новом класом уместо проширивањем централног условног израза.

---

### НАПОМЕНА

Исти принцип раздвајања одговорности примењује се и у пословном коду.

# IToolGuardrail и GuardrailEvaluator

Мала и проверљива правила олакшавају тестирање и проширење политика.

## КОД

```
1 public interface IToolGuardrail
2 {
3     GuardrailDecision Evaluate(ToolInvocation invocation);
4 }
5
6 public sealed class GuardrailEvaluator(IEnumerable<IToolGuardrail> guardrails)
7 {
8     {
9         public GuardrailDecision Evaluate(ToolInvocation
10             invocation)
11         {
12             foreach (var guardrail in guardrails)
13                 if (!guardrail.Evaluate(invocation).Allowed) return
14                     decision;
15             return GuardrailDecision.Allow();
16         }
17     }
```

## НАПОМЕНЕ

Evaluator зависи од апстракције.

Нова политика је нова класа.

Централни ток остаје стабилан.

## Детерминистичке guardrail политике

Свака политика има јасан домет и стабилан исход Allow или Block.

01

### **SensitiveFile**

блокира .env и secrets.json

02

### **DangerousCommand**

блокира force push и rm -rf

03

### **Allowlist**

још јача граница за ризичне системе

04

### **Decision**

стабилно Allow/Block понашање

# SensitiveFileGuardrail

Правило блокира приступ познатим датотекама са тајним подацима.

## КОД

```
1 public sealed class SensitiveFileGuardrail :  
2     IToolGuardrail  
3 {  
4     private static readonly string[] ForbiddenNames =  
5         [".env", "secrets.json", "appsettings.secrets.json"];  
6  
7     public GuardrailDecision Evaluate(ToolInvocation  
8         invocation)  
9     {  
10         var normalized = invocation.FilePath?.Replace("\\",  
11             "/");  
12         return ForbiddenNames.Any(name => normalized?.  
13             EndsWith(name) == true)  
14             ? GuardrailDecision.Block("Sensitive file blocked.")  
15             : GuardrailDecision.Allow();  
16     }  
17 }
```

## НАПОМЕНЕ

Тајне не смеју постати context.

Путања се нормализује.

Одлука је детерминистичка.

# DangerousCommandGuardrail

Деструктивне команде блокирају се пре извршења.

```
1 private static readonly string[] ForbiddenFragments =  
2 [  
3     "git push --force",  
4     "git push -f",  
5     "rm -rf",  
6     "Remove-Item -Recurse -Force",  
7     "format c:"  
8 ];
```

## НАПОМЕНЕ

Ово је denylist приступ.

За ризичније системе allowlist је јача граница.

Политика мора бити тестирана.

## Hook као адаптер политике

PreToolUse повезује конкретан AI алат са guardrail политиком.

**Сама guardrail апликација остаје обичан .NET пројекат у solution-у и може се повезати са другим клијентом другим адаптер-ом.**

---

### НАПОМЕНА

**Адаптер се мења, политика остаје.**

## .claude/settings.json као адаптер

Hook позива guardrail пројекат пре ризичне операције.

### КОД

```
1 {
2   "hooks": {
3     "PreToolUse": [
4       {
5         "matcher": "Bash|Read|Edit|Write",
6         "hooks": [{
7           "type": "command",
8           "command": "dotnet run --project
9             src/EquipmentReservation.Guardrails/Equipment
10             Reservation.Guardrails.csproj"
11         }]
12       }
13     ]
14   }
15 }
```

### НАПОМЕНЕ

Конфигурација је спољни детаљ.

Политика остаје тестабилна.

Исти принцип граница важи као у Clean Architecture.

## Тестирање guardrail политике

Guardrail политика проверава се аутоматизованим тестовима као и други код.

**Исти EquipmentReservation.Tests пројекат проверава да ризичне команде и приступ .env датотеци буду одбијени.**

---

### НАПОМЕНА

Ефикасност заштите потврђују аутоматизовани тестови.

# Unit тестови guardrail политика

Unit тестови потврђују да забране и дозволе имају очекивано понашање.

```
1 [TestCase("git push --force origin main")]
2 [TestCase("rm -rf ./src")]
3 public void DangerousCommandGuardrail_BlocksDestructiveComm
4     ands(string command)
5 {
6     var guardrail = new DangerousCommandGuardrail();
7     var result = guardrail.Evaluate(new ToolInvocation(
8         "Bash", command, null));
9     Assert.That(result.Allowed, Is.False);
10 }
11
12 [Test]
13 public void SensitiveFileGuardrail_BlocksEnvFile()
14 {
15     var result = guardrail.Evaluate(new ToolInvocation(
16         "Read", null, "/repo/.env"));
17     Assert.That(result.Allowed, Is.False);
18 }
```

## НАПОМЕНЕ

Тестови су брзи и детерминистички.

Политика се проверава без AI модела.

Ово је нормалан code quality механизам.

# Јединствена провера пословних и guardrail тестова

Једна команда извршава све тестове у solution-у.



```
1 dotnet test EquipmentReservation.sln --configuration
2 Release
```

## НАПОМЕНЕ

Исти solution је извор истине.

Пословни и tooling тестови пролазе заједно.

CI може да проверава обе врсте правила.

## Евалуациони сценарији и unit тестови

Unit тестови проверавају детерминистичке политике, а eval сценарији понашање агентског тока.

**Eval сценарио проверава цело агентско понашање у условима који личе на стварни ток рада: да ли модел поштује инструкције, користи алате правилно и не тврди непроверене резултате.**

---

### НАПОМЕНА

Unit тестови и eval сценарији покривају различите нивое провере.

## Сврха unit тестова и eval сценарија

Оба механизма су потребна и покривају различите ризике.

| Провера          | Шта доказује                                            |
|------------------|---------------------------------------------------------|
| Unit тест        | конкретна политика враћа Allow/Block како је дефинисано |
| Integration test | solution пролази кроз стварне команде                   |
| Eval scenario    | agentski ток поштује контекст, ограничења и доказе      |
| Human review     | тим прихвата или одбија резултат према критеријумима    |

# Сигнали завршног QA прегледа

Завршни преглед повезује архитектуру, тестове, AI траг и diff.

**01**

## Архитектура

границе и dependency rule

**02**

## Тестови

пословни и guardrail сценарији

**03**

## AI trag

AI\_USAGE и одлуке тима

**04**

## Diff

мали, објашњив и релевантан

## Неприхватљиви поступци

Guardrails не могу надокнадити неразумевање архитектуре, тестова и пословних правила.

### ЛОШЕ

model тврди да је тестирао без команде

чита .env или secrets

force push у дељеном току

мења тест да сакрије грешку

### ДОБРО

реална build/test провера

блокирани ризични алати

јасан AI\_USAGE запис

review пре прихватања

## Задатак на вежби

Једно правило из инструкција претвара се у извршиву и тестирану guardrail политику.

- 01      Покренути guardrail тестове.
- 02      Проверити да .env приступ буде блокиран.
- 03      Проверити да деструктивне команде буду блокиране.
- 04      Објаснити разлику између hook адаптер-а и саме политике.
- 05      Осмислити један eval сценарио за agentski ток.

## Резиме

Обавезно правило се имплементира, тестира и укључује у завршни QA поступак.

**Када је правило важно, треба га претворити у код, тестирати као сваки други део система и укључити у завршни QA ток.**

---

### НАПОМЕНА

Завршни QA повезује правила, тестове, евалуације и одбрану.

## Захтеви за завршну одбрану

Студент повезује захтеве, архитектуру, тестове, AI радни ток, MCP и guardrails.

- 01 Показати захтев, backlog ставку и критеријуме.
- 02 Објаснити архитектонске границе и use-case ток.
- 03 Показати тестове и резултат покретања.
- 04 Показати AI\_USAGE и контролисан AI workflow.
- 05 Објаснити MCP/guardrail део без мешања са пословним језгром.